极客时间

内容输出与分发

从 JSON 到用户可读内容 — 最后一公里

第 4 周 · 第 14 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

V2 输出 JSON
用户需要可读内容

采集 → 分析 → 存储 → ??? → 用户

分发三层架构

极客时间

数据层
(Data)

knowledge

articles

*.json

按 score 排序

格式化层
(Formatter)

formatter.py

to_markdown()

to_telegram()

to_feishu()

推送层
(Publisher)

publisher.py

publish_telegram()

publish_feishu()

批结构化知识

publish_file()

每层独立变化，新增渠道不改核心代码

极客时间黄佳老师出品

为什么三层解耦?

极客时间

问题 1：新增渠道要改核心代码

- 加个邮件推送 → 改推送函数 → 影响 Telegram/飞书

问题 2：格式和推送耦合

- Telegram 格式变了 → 飞书推送代码也得看一遍

问题 3：测试困难

- 想测格式化效果 → 必须连推送一起跑 → 需要真实 API

- **解耦后：** 格式化层纯函数，零外部依赖，可独立测试

极客时间

极客时间

格式化层实战

formatter.py — Markdown / Telegram / 飞书卡片

极客时间黄佳老师出品

数据加载 — load_articles()

极客时间

formatter.py | python

```python
import json, glob, os
from datetime import datetime

def load_articles(data_dir='knowledge/articles'):
    """从 JSON 文件加载文章，按 relevance_score 降序"""
    articles = []
    for path in glob.glob(os.path.join(data_dir, '*.json')):
        with open(path, 'r', encoding='utf-8') as f:
            data = json.load(f)
            if isinstance(data, list):
                articles.extend(data)
            else:
                articles.append(data)
    articles.sort(
        key=lambda a: a.get('relevance_score', 0),
        reverse=True
    )
    return articles
```

Markdown 周报 — to_markdown_digest()

formatter.py | python

```python
def to_markdown_digest(articles, top_n=10):
    """生成 Markdown 格式的周报摘要"""
    today = datetime.now().strftime('%Y-%m-%d')
    lines = [
        f'# AI 技术周报 — {today}',
        '',
        f'> 本期精选 {min(top_n, len(articles))} 篇高价值内容',
        '', '---', '',
    ]
    for i, art in enumerate(articles[:top_n], 1):
        score = art.get('relevance_score', 0)
        tags = ', '.join(art.get('tags', []))
        lines.extend([
            f'## {i}. {art.get("title", "无标题")}',
            '',
            f'**来源**: {art.get("source", "未知")} | '
            f'**相关度**: {score}/100 | **标签**: {tags}',
            '',
            f'> {art.get("summary", "暂无摘要")}',
            '',
            f'[阅读原文]({art.get("url", "#")})',
            '', '---', '',
        ])
    return '\n'.join(lines)
```

Telegram 消息 — to_telegram_message()

极客时间

formatter.py | python

```python
def to_telegram_message(articles, top_n=5):
    """生成 Telegram Bot 推送消息（限 4096 字符）"""
    today = datetime.now().strftime('%m/%d')
    lines = [f'AI 日报 {today}', '']

    for i, art in enumerate(articles[:top_n]):
        score = art.get('relevance_score', 0)
        title = art.get('title', '无标题')
        summary = art.get('summary', '')
        if len(summary) > 80:
            summary = summary[:77] + '...'
        url = art.get('url', '')

        lines.append(f'{i+1}. {title} [{score}分]')
        lines.append(f'   {summary}')
        if url:
            lines.append(f'   {url}')
        lines.append('')

    message = '\n'.join(lines)
    if len(message) > 4000:
        message = message[:3997] + '...'
    return message
```

极客时间黄佳老师出品

飞书互动卡片 — to_feishu_card()

formatter.py | python

```python
def to_feishu_card(articles, top_n=5):
    """生成飞书互动卡片 JSON"""
    today = datetime.now().strftime('%Y-%m-%d')
    elements = []
    for i, art in enumerate(articles[:top_n], 1):
        score = art.get('relevance_score', 0)
        md = (f'**{i}. {art["title"]}** ({score}/100)\n'
              f'{art.get("summary", "")}\n'
              f'标签: {" | ".join(art.get("tags", []))}')
        elements.append({'tag': 'markdown', 'content': md})
        if art.get('url'):
            elements.append({'tag': 'action', 'actions': [
                {'tag': 'button', 'type': 'primary',
                 'text': {'tag': 'plain_text', 'content': '阅读原文'},
                 'url': art['url']}
            ]})
        elements.append({'tag': 'hr'})
    return {'msg_type': 'interactive', 'card': {
        'header': {'title': {'tag': 'plain_text',
            'content': f'AI 技术日报 — {today}'}, 'template': 'blue'},
        'elements': elements}}
```

三种输出格式对比

维度	Markdown	Telegram	飞书卡片
用途	博客/笔记	即时通讯	企业协作
信息量	高（完整摘要）	中（截断摘要）	高（摘要+按钮）
交互性	无	无	有（按钮跳转）
字符限制	无	4096 字符	30KB
推荐条数	10	5	5

极客时间

推送层实战

publisher.py — Telegram Bot API / 飞书 Webhook / 本地文件

极客时间黄佳老师出品

Telegram Bot API 推送

publisher.py | python

```python
import os, requests
from dotenv import load_dotenv
load_dotenv()

def publish_telegram(message):
    """通过 Telegram Bot API 发送消息"""
    token = os.getenv('TELEGRAM_BOT_TOKEN')
    chat_id = os.getenv('TELEGRAM_CHAT_ID')
    if not token or not chat_id:
        print('[WARN] Telegram 环境变量未设置')
        return {'ok': False, 'reason': 'env not set'}

    url = f'https://api.telegram.org/bot{token}/sendMessage'
    payload = {
        'chat_id': chat_id,
        'text': message,
        'parse_mode': 'HTML',
        'disable_web_page_preview': True,
    }
    try:
        resp = requests.post(url, json=payload, timeout=10)
        return resp.json()
    except Exception as e:
        return {'ok': False, 'reason': str(e)}
```

飞书 Webhook 推送

极客时间

publisher.py | python

```python
def publish_feishu(card):
    """通过飞书 Webhook 发送互动卡片"""
    webhook_url = os.getenv('FEISHU_WEBHOOK_URL')
    if not webhook_url:
        print('[WARN] FEISHU_WEBHOOK_URL 未设置')
        return {'ok': False, 'reason': 'env not set'}

    try:
        resp = requests.post(webhook_url, json=card, timeout=10)
        result = resp.json()
        if result.get('code') == 0:
            print('[OK] 飞书推送成功')
        else:
            print(f'[ERROR] 飞书推送失败: {result.get("msg")}')
        return result
    except Exception as e:
        return {'ok': False, 'reason': str(e)}


def publish_file(content, path='output/digest.md'):
    """导出到本地文件"""
    from pathlib import Path
    output = Path(path)
    output.parent.mkdir(parents=True, exist_ok=True)
    output.write_text(content, encoding='utf-8')
    print(f'[OK] 文件已导出: {output.resolve()}')
    return str(output.resolve())
```

环境变量配置 — .env

极客时间

.env | bash

```bash
# .env — 推送渠道配置

# Telegram Bot
TELEGRAM_BOT_TOKEN=123456:ABC-DEF1234ghIkl-zyx57W2v1u123ew11
TELEGRAM_CHAT_ID=-1001234567890

# 飞书 Webhook
FEISHU_WEBHOOK_URL=https://open.feishu.cn/open-apis/bot/v2/hook/xxx

# ────────────────────────────────────────────────────────────
# Telegram Bot 创建步骤：
#   1. 搜索 @BotFather → /newbot
#   2. 获取 Token
#   3. 拉 Bot 入群
#   4. getUpdates 获取 chat_id
# ────────────────────────────────────────────────────────────
# 飞书 Webhook 创建步骤：
#   群设置 → 群机器人 → 自定义机器人
#   复制 Webhook 地址
```

统一分发入口 — distribute()

极客时间

publisher.py | python

```python
def distribute(articles, channels=None):
    """一键推送到多个渠道"""
    from formatter import (
        to_markdown_digest, to_telegram_message, to_feishu_card
    )
    if channels is None:
        channels = ['markdown', 'telegram', 'feishu']

    results = {}
    if 'markdown' in channels:
        md = to_markdown_digest(articles, top_n=10)
        results['markdown'] = {'ok': True,
            'path': publish_file(md)}
    if 'telegram' in channels:
        msg = to_telegram_message(articles, top_n=5)
        results['telegram'] = publish_telegram(msg)
    if 'feishu' in channels:
        card = to_feishu_card(articles, top_n=5)
        results['feishu'] = publish_feishu(card)

    print(f'分发完成: {len(results)} 个渠道')
    return results
```

极客时间黄佳老师出品

OpenClaw Skill 集成

daily-digest Skill + Cron 定时推送

极客时间

daily-digest Skill + Cron 配置

skills/daily-digest/SKILL.md | yaml

```yaml
# skills/daily-digest/SKILL.md
---
name: daily-digest
description: 生成并推送每日 AI 技术摘要
allowed-tools:
  - Read
  - Write
  - Bash
---

# Daily Digest Skill

## 任务
1. 加载 knowledge/articles/ 最新文章
2. 生成 Markdown / Telegram / 飞书 三种格式
3. 推送到配置的渠道

## Cron 配置
openclaw cron add --skill daily-digest \
  --schedule "0 8 * * *" --name "每日早报"

# 手动测试
openclaw run daily-digest
```

本节作业

极客时间

任务 1：实现 formatter 格式化模块

编写 formatter.py，实现 load_articles()、to_markdown_digest()、to_telegram_message()、to_feishu_card()。用测试 JSON 数据验证三种格式输出。

任务 2：实现 publish 推送模块

编写 publisher.py，实现 publish_telegram()、publish_feishu()、publish_file()。配置 .env，测试真实推送（至少一个渠道）。

任务 3：配置每日推送 Skill

创建 skills/daily-digest/SKILL.md。配置 Cron 定时任务（openclaw cron add 或 crontab）。测试自动推送流程。

极客时间黄佳老师出品

本节小结

极客时间

工程思想：分发三层解耦

数据层 / 格式化层 / 推送层各自独立，新增渠道不改核心代码

formatter.py：

三种输出格式（Markdown 全量 / Telegram 4096 限制 / 飞书互动卡片）

publisher.py：并发推送到多渠道，失败不阻塞其他渠道

daily-digest Skill + Cron：每日自动推送，从数据到用户全自动

关键原则：

渠道无关的核心逻辑下沉到 formatter，渠道感知逻辑封在 publisher

下一节：第 15 节 读者互动 Bot — 让用户查询知识库